Malaviya National Institute of Technology Jaipur
Department of Computer Science and Engineering

IoT Device Management System - Vulnerabilities and SAST

Application 10 (group 10 mod 10 → IoT Device Management), Python + Semgrep

Course	Cryptography Laboratory (22CPP307)
Assignment	Lab Assignment 3 (Secure Application)
Group	Group 10 (Even group number)
Team members	Rishi Saini (2024UCP1566), Nandini Verma (2024UCP1667)
Instructor	Dr. Meenakshi Tripathi
Submission date	15 October 2026
Folder	secure_application/
SAST tool	Semgrep 1.181

1. Application

A console application that simulates an IoT device-management back end. Core functionalities: **(1)** smart-device registration, **(2)** device-status monitoring (ping), **(3)** firmware upload and viewing, **(4)** configuration management, **(5)** device listing. Python was chosen because Semgrep is the SAST tool installed.

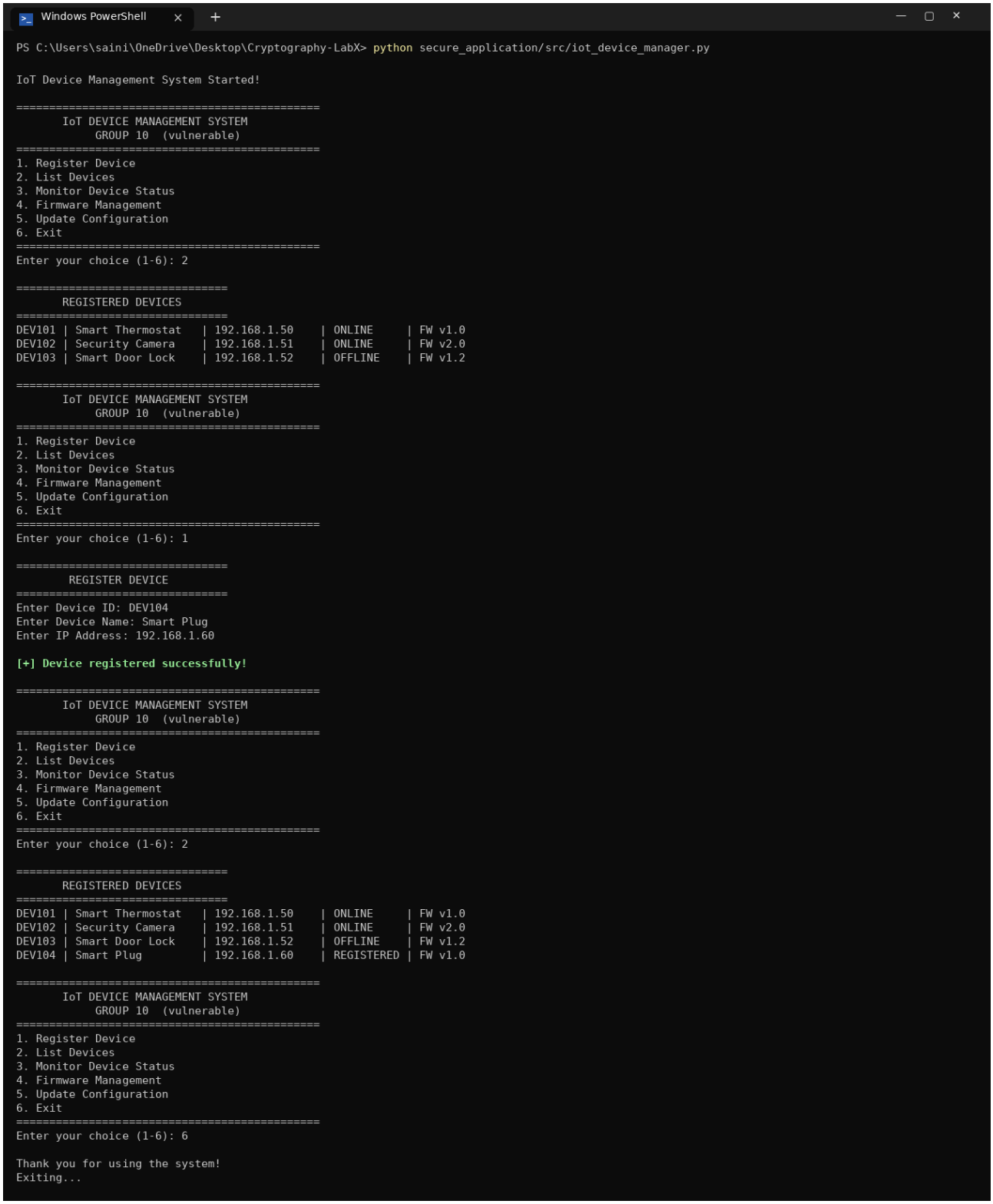


Figure 1 - Application menu, device registration and listing

2. The three intentional vulnerabilities

ID	Vulnerability (CWE)	Where	Exploit used	Impact
V1	Command Injection (CWE-78)	build_ping_command() / monitor_device_core()	IP = 127.0.0.1 & echo INJECTED_BY_ATTACKER	Arbitrary OS command execution on the server

ID	Vulnerability (CWE)	Where	Exploit used	Impact
V2	Path Traversal (CWE-22)	read_firmware()	filename = ../internal/device_keys.txt	Disclosure of files outside firmware_store/
V3	Missing Authentication (CWE-306)	update_configuration(), upload_firmware()	No login needed to change status/firmware	Anyone can reconfigure or replace firmware of any device

V1 - Command injection

```
command = 'ping -c 1 ' + ip                                    # ip is user supplied
subprocess.run(command, shell=True, capture_output=True, text=True)   # shell interprets & ; |
```

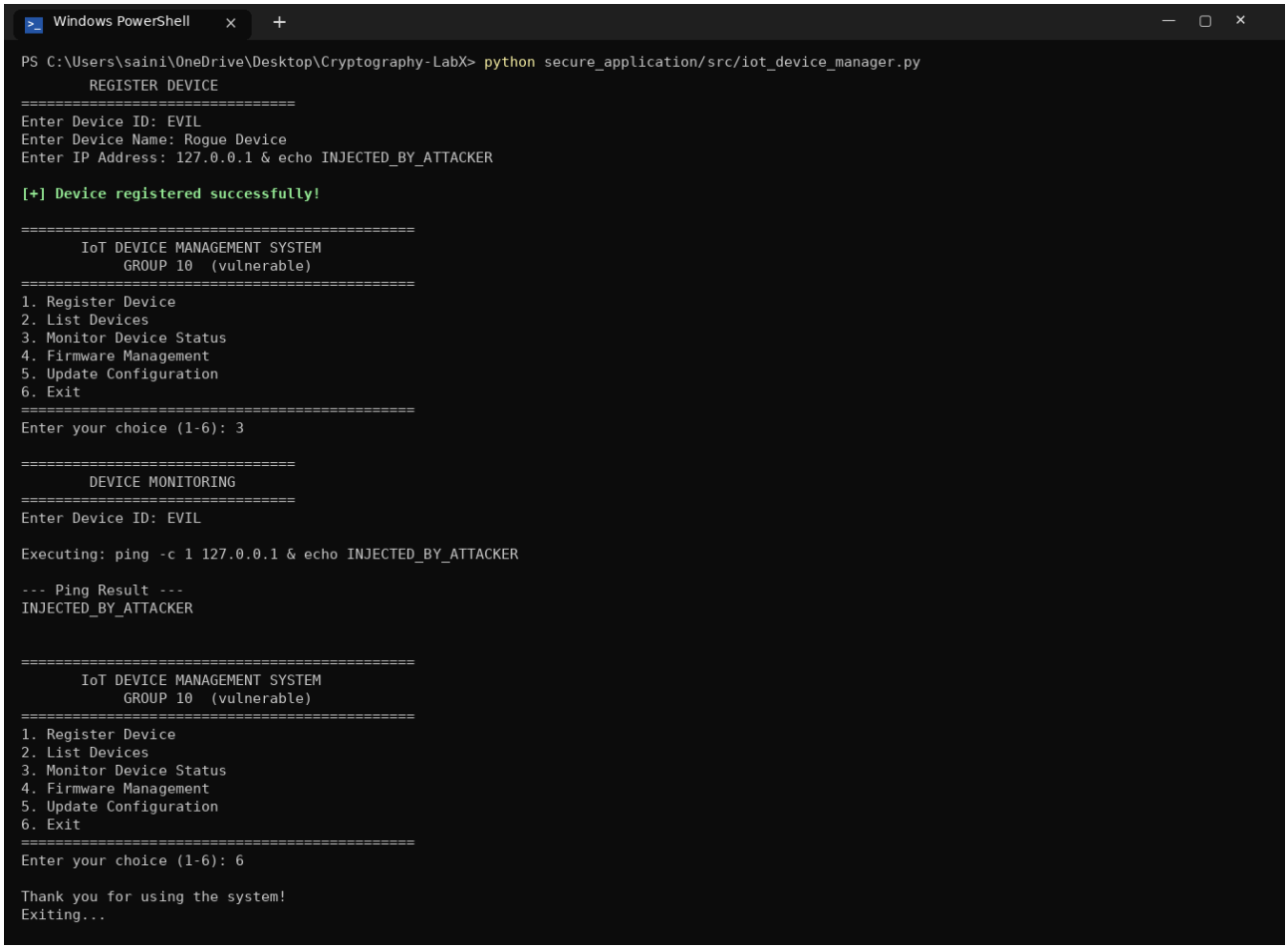


Figure 2 - Injected command executes

V2 - Path traversal

```
file_path = os.path.join(FIRMWARE_FOLDER, filename)   # '../internal/device_keys.txt' escapes the folder
with open(file_path) as handle: return handle.read()
```

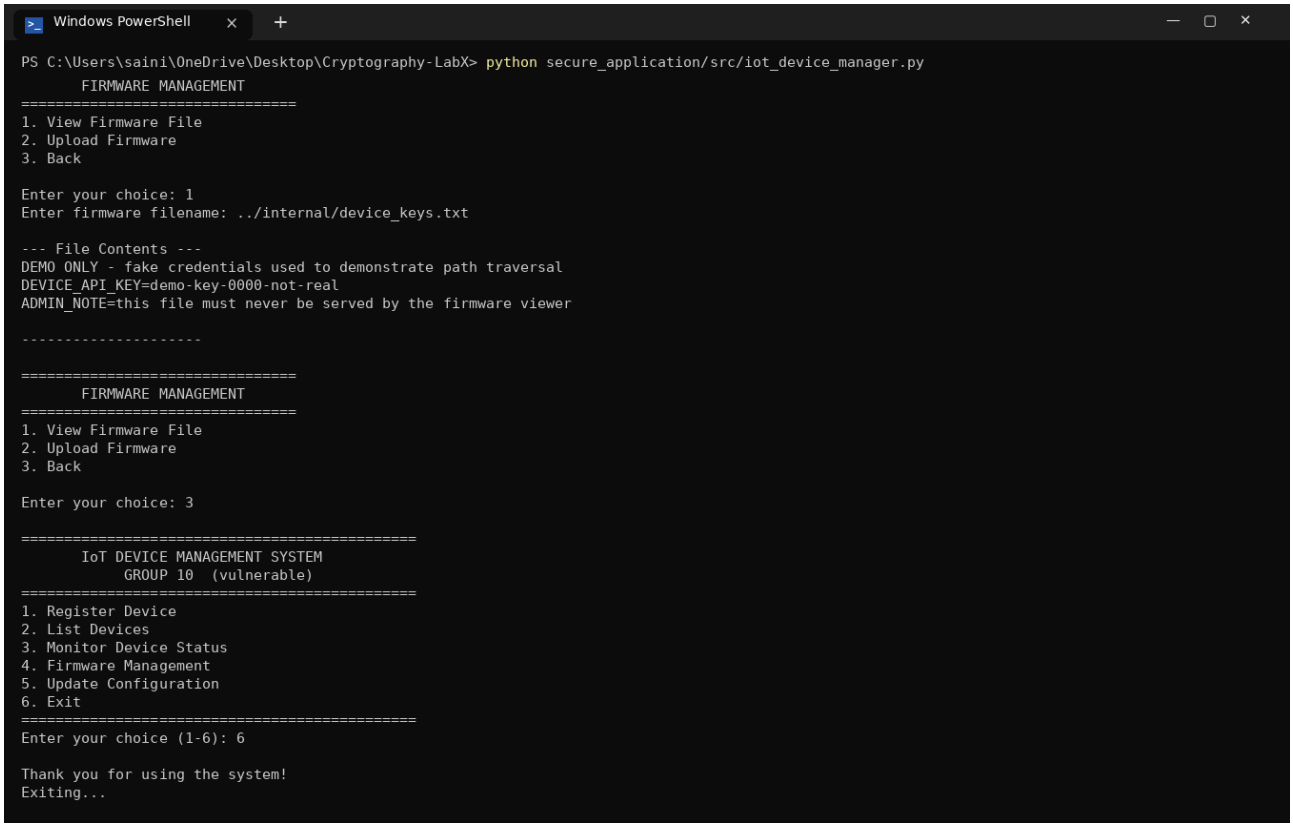


Figure 3 - Fake credentials file outside the firmware folder is disclosed

V3 - Missing authentication

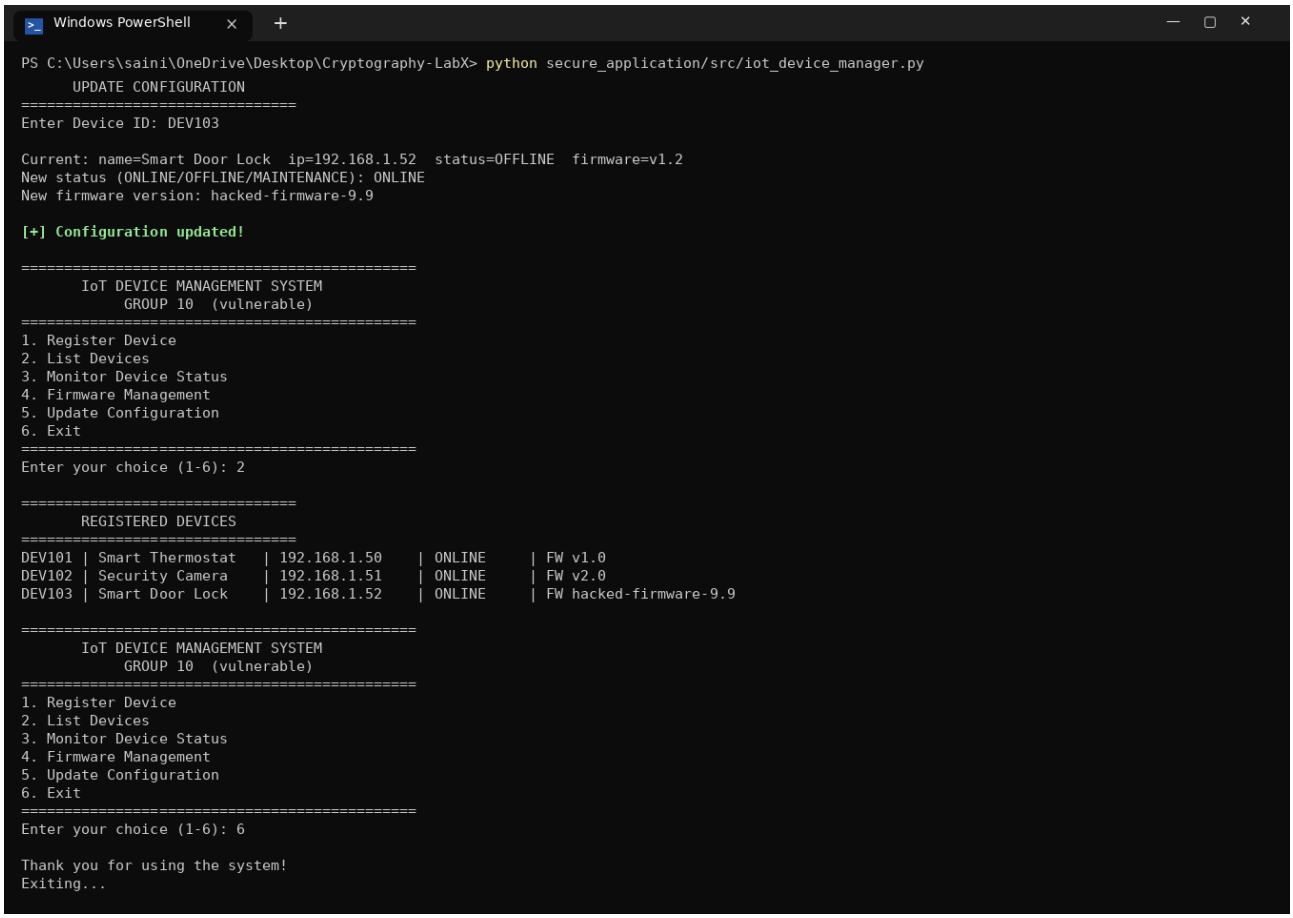


Figure 4 - Device firmware changed with no login

3. SAST with Semgrep

Semgrep was run on src/ with the lab's custom rules (sast/rules/iot_rules.yml, one rule per vulnerability). It reported **5 findings, all in the vulnerable file**; the hardened file is clean. Reports: reports/sast_report.txt and reports/sast_report.json. Run `python secure_application/sast/run_scan.py --registry` to add the public Semgrep rule-sets.

Rule	Line	Function	Vulnerability
iot-v1-command-injection-shell-true	66	monitor_device_core	V1
iot-v2-path-traversal-open-joined-path	73	read_firmware	V2
iot-v2-path-traversal-open-joined-path	81	upload_firmware	V2
iot-v3-missing-authentication-privileged-function	81	upload_firmware	V3
iot-v3-missing-authentication-privileged-function	94	update_configuration	V3

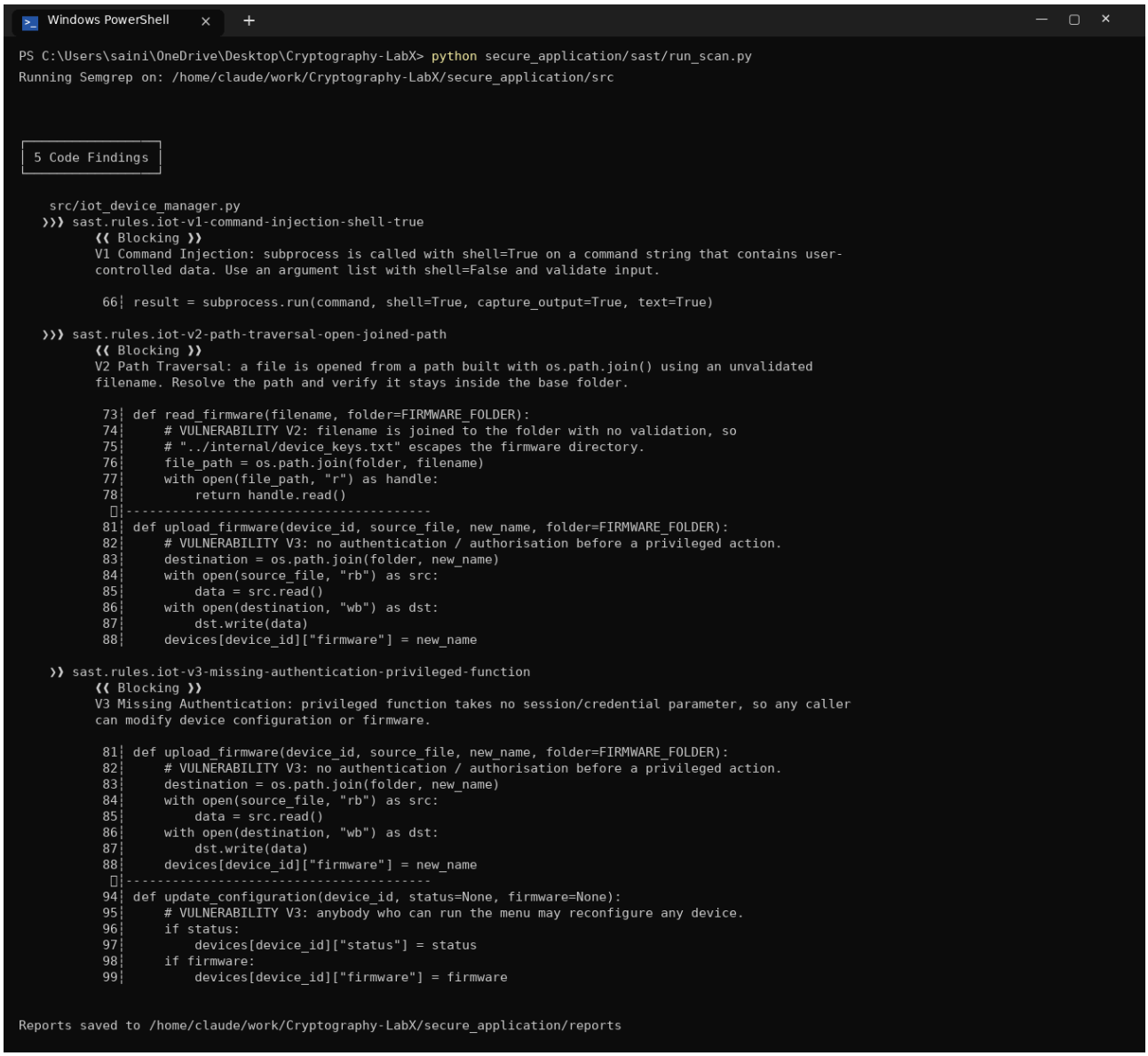


Figure 5 - Semgrep scan output

Limitation: static rules find code patterns, not logic flaws; the missing-authentication rule only flags privileged functions that take no session parameter, so every finding was confirmed manually by the exploits above.

4. Remediation (iot_device_manager_secure.py)

Vulnerability	Fix
V1	Validate with the ipaddress module on registration and before pinging; call subprocess.run(['ping','-c','1',ip], shell=False).
V2	Resolve the path with realpath(), require commonpath() to equal the firmware folder, allow only .txt/.bin/.fw files.
V3	Salted PBKDF2-HMAC-SHA256 login (200,000 rounds, constant-time compare); privileged functions require a Session object; status values allow-listed.

5. Tests

18 unit tests: 5 functional, 3 that prove the vulnerabilities, 10 that prove the fixes. All pass.

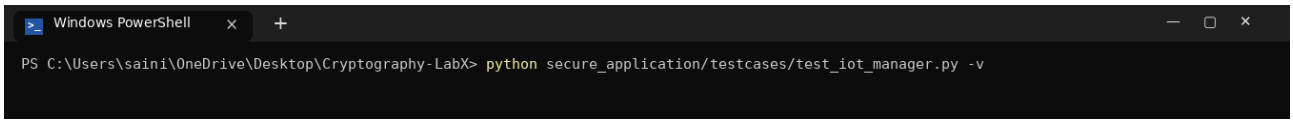


Figure 6 - Test results

6. Conclusion

The application demonstrates three exploitable weaknesses that map to the OWASP Top 10 injection and access-control classes, shows how a SAST tool finds the pattern-based ones, and provides a verified hardened version.